Advanced Predator-Prey Simulation- Complete Walkthrough

Table of Contents

1. Project Overview
2. Dependencies and Imports
3. Simulation Parameters Explained
4. Code Structure and Classes
5. Main Simulation Loop
6. Key Algorithms and Behaviors
7. User Controls and Features
8. Data Visualization
9. Running the Simulation

---

1. Project Overview

What This Project Does

This is an **Advanced Predator-Prey Ecosystem Simulation** that models a complex biological system where:

- **Prey** (green circles) must find food, avoid predators, and reproduce
- **Predators** (red triangles) hunt prey to survive and reproduce
- **Food** (yellow dots) spawns randomly and provides energy
- **Obstacles** (gray circles) create environmental challenges

The simulation demonstrates **emergent behavior** through simple rules, creating realistic population dynamics similar to the Lotka-Volterra predator-prey model.

Key Features

- **Energy System**: Agents lose energy over time and must eat to survive
- **Reproduction System**: Agents can mate and produce offspring
- **Flocking Behavior**: Prey use the Boids algorithm to form defensive herds
- **Real-time Visualization**: Live population graphs and statistics
- **Interactive Controls**: Adjust parameters during simulation

---

2. Dependencies and Imports

Lines 1-6: Import statements

```
import pygame          # Graphics and game loop

import random          # Random number generation

import math            # Mathematical functions (cos, sin, etc.)

import os              # File system operations

import matplotlib.pyplot as plt  # Graph generation

from collections import deque   # (Note: imported but not used in current
code)
```

- **pygame**: Handles all rendering, event processing, and the main game loop
- **random**: Used for random positions, angles, and food spawning
- **math**: Used for trigonometry when calculating velocities from angles
- **matplotlib**: Generates high-quality graphs when simulation ends or when 'G' is pressed

---

3. Simulation Parameters Explained

All parameters are defined at the top of the file (lines 8-64) for easy tuning.

Screen Configuration (Lines 12-14)

```
WIDTH, HEIGHT = 800, 600  # Window size in pixels

FPS = 60                   # Frames per second (simulation speed)
```

- Controls the display window size and simulation update rate

Visual Colors (Lines 16-22)

```
BACKGROUND_COLOR = (30, 30, 30)      # Dark gray background

PREY_COLOR = (0, 255, 0)             # Green for prey
```

```
PREDATOR_COLOR = (255, 0, 0)          # Red for predators

FOOD_COLOR = (255, 255, 0)            # Yellow for food

OBSTACLE_COLOR = (128, 128, 128)      # Gray for obstacles

TEXT_COLOR = (200, 200, 200)          # Light gray for UI text
```

- RGB color tuples used for rendering different entities

Initial Population (Lines 24-28)

```
INITIAL_PREY = 50          # Starting number of prey

INITIAL_PREDATORS = 3      # Starting number of predators

INITIAL_FOOD = 20          # Starting food pellets

INITIAL_OBSTACLES = 5      # Starting obstacles
```

- Sets the initial state of the ecosystem when simulation starts

Agent Physics (Lines 30-36)

```
PREY_SPEED = 2.0              # Pixels per frame for prey

PREDATOR_SPEED = 2.6           # Pixels per frame for predators (faster!)

PREY_VISION_RADIUS = 50       # How far prey can see predators/food

PREDATOR_VISION_RADIUS = 100  # How far predators can see prey

OBSTACLE_RADIUS = 30          # Size of obstacles

FOOD_RADIUS = 3               # Size of food pellets
```

- **Speed**: Higher values = faster movement
- **Vision Radius**: Detection range for nearby entities
- **Radii**: Collision detection sizes

Energy System (Lines 38-44)

```
ENERGY_LIFETIME_FRAMES = 1500  # Frames until death without food

MAX_ENERGY = 100.0             # Maximum energy value

ENERGY_LOSS_RATE = MAX_ENERGY / ENERGY_LIFETIME_FRAMES  # ~0.067 per frame

FOOD_ENERGY_GAIN = 30.0        # Energy gained from eating food

PREDATOR_PREY_ENERGY_GAIN = 50.0  # Energy gained from eating prey

LOW_ENERGY_THRESHOLD = 40.0    # Below this, prey prioritizes food
```

- **ENERGY_LIFETIME_FRAMES**: At 60 FPS, agents die after ~25 seconds without food
- **ENERGY_LOSS_RATE**: Automatically calculated to deplete energy over lifetime
- **LOW_ENERGY_THRESHOLD**: When energy < 40%, prey stops flocking/mating and seeks food

Reproduction System (Lines 46-50)

```
REPRODUCTION_ENERGY_THRESHOLD = 80.0    # Must have >80% energy to mate

REPRODUCTION_COOLDOWN_FRAMES = 200      # ~3.3 seconds between matings

MATING_DURATION_FRAMES = 60             # ~1 second mating ritual

MATING_DISTANCE = 20.0                  # Must be within 20 pixels to mate
```

- **Threshold**: Prevents weak agents from reproducing
- **Cooldown**: Prevents population explosion
- **Duration**: Agents are vulnerable (immobile) during mating
- **Distance**: How close agents must be to start mating

Flocking Behavior - Boids Algorithm (Lines 52-58)

```
FLOCKING_ENABLED = True          # Toggle for flocking behavior

FLOCKING_RADIUS = 50             # How far to look for neighbors

ALIGNMENT_WEIGHT = 0.5           # Strength of "match direction" rule

COHESION_WEIGHT = 0.3            # Strength of "move toward center" rule
```

```
SEPARATION_WEIGHT = 0.8          # Strength of "avoid crowding" rule

FLOCK_SPEED_BOOST = 0.05         # Speed increase per neighbor
```

- **FLOCKING_RADIUS**: Only nearby prey influence each other
- **Weights**: Control the balance between the three Boids rules
- **SPEED_BOOST**: Larger flocks move slightly faster (safety in numbers)

Food Spawning (Lines 60-61)

```
FOOD_SPAWN_CHANCE = 0.05  # 5% chance per frame to spawn new food
```

- At 60 FPS, this averages ~3 food pellets per second
- Can be adjusted in real-time with keys 2/3

History Tracking (Lines 63-64)

```
HISTORY_MAX_LEN = 1000  # (Note: Currently unlimited, this is unused)
```

- Originally intended to limit history, but currently tracks all frames

---

4. Code Structure and Classes

4.1 Initialization (Lines 66-74)

```
pygame.init()

screen = pygame.display.set_mode((WIDTH, HEIGHT))

pygame.display.set_caption("Advanced Predator-Prey Simulation")

clock = pygame.time.Clock()

font = pygame.font.SysFont(None, 24)
```

- Initializes Pygame, creates the window, sets up the clock for FPS control, and creates a font for text rendering

4.2 Food Class (Lines 80-89)

Purpose: Represents food pellets that provide energy

Key Methods:

- `__init__(x, y)` : Creates food at specified position or random location
- `draw()` : Renders a yellow circle on screen

Code Flow:

```
class Food:
    def __init__(self, x=None, y=None):
        # If no position given, spawn randomly
        self.position = pygame.math.Vector2(...)
        self.radius = FOOD_RADIUS  # 3 pixels
```

4.3 Obstacle Class (Lines 91-100)

Purpose: Static obstacles that agents must avoid

Key Methods:

- `__init__(x, y)` : Creates obstacle at position
- `draw()` : Renders a gray circle

Behavior: Agents detect obstacles within 30 pixels and steer away (see `Agent.avoid_obstacles()`)

4.4 Agent Base Class (Lines 102-148)

Purpose: Base class providing common functionality for Prey and Predators

Attributes (Lines 103-119):

```
self.position      # Current location (Vector2)
self.velocity      # Direction and speed (Vector2, normalized)
```

```
self.base_speed    # Base movement speed
self.speed         # Current speed (can be modified by flocking)
self.color         # Visual color
self.radius        # Collision radius (4 pixels)
self.energy        # Current energy (0-100)
self.trail         # List of recent positions for trail effect
self.reproduction_cooldown  # Frames until can mate again
self.mating_timer  # Frames remaining in mating ritual
self.mating_partner # Reference to partner during mating
self.alive         # Boolean: is agent alive?
```

Key Methods:

1. `update_energy()` (Lines 121-124):
   - Decreases energy by `ENERGY_LOSS_RATE` each frame
   - Sets `alive = False` when energy reaches 0
2. `check_bounds()` (Lines 126-132):
   - Bounces agents off screen edges
   - Prevents agents from leaving the simulation area
3. `update_trail()` (Lines 134-137):
   - Stores last 10 positions for visual trail effect
   - Creates a "tail" showing movement history
4. `draw_trail()` (Lines 139-141):
   - Draws a line connecting recent positions
5. `avoid_obstacles()` (Lines 143-148):
   - Checks distance to all obstacles
   - If within avoidance radius (obstacle radius + 30), steers away

- Adds repulsion force to velocity

4.5 Prey Class (Lines 150-275)

Purpose: The most complex agent type with multiple competing behaviors

Initialization (Lines 151-153):

```
def __init__(self, x=None, y=None):
    super().__init__(x, y, PREY_SPEED, PREY_COLOR)
    self.vision_radius = PREY_VISION_RADIUS  # 50 pixels
```

Update Method - Priority System (Lines 155-232):

The `update()` method implements a **priority-based behavior system**:

1. **Early Returns** (Lines 156-168):

   - If dead, return immediately

   - Update energy and cooldown timers

   - If mating, stop moving and return

2. **Priority 1: Flee from Predators** (Lines 170-181):

   ```python
   # Find nearest predator within vision radius
   # If found, add flee direction to velocity
   flee_dir = (self.position - nearest_predator.position).normalize()
   self.velocity += flee_dir * 0.5
   ```

   - **Highest priority**: Survival over everything else

   - Steers directly away from nearest predator

3. **Priority 2: Seek Food** (Lines 183-195):

   ```python
   elif self.energy < LOW_ENERGY_THRESHOLD:  # < 40%
```

```
    # Find nearest food
    # Steer toward it
```

- Only activates when energy is low
- Overrides other behaviors (except fleeing)

4. **Priority 3: Seek Mate** (Lines 197-217):

```python
elif self.energy > REPRODUCTION_ENERGY_THRESHOLD:  # > 80%
    # Find eligible mate (high energy, no cooldown, no partner)
    # If close enough, start mating ritual
```

- Only when energy is high
- Both partners must meet requirements
- Starts mating timer when within `MATING_DISTANCE`

5. **Priority 4: Flocking** (Lines 219-221):

- Always applies if enabled
- Uses Boids algorithm (see below)

6. **Priority 5: Obstacle Avoidance** (Line 224):

- Always active

7. **Final Steps** (Lines 226-232):

- Normalize velocity to unit vector
- Update position: `position += velocity * speed`
- Check boundaries
- Update trail

Flocking Algorithm - `apply_flocking()` (Lines 234-268):

Implements the **Boids Algorithm** with three rules:

1. **Alignment** (Lines 244, 252-253):

```python
alignment += other.velocity  # Sum all neighbor velocities
alignment = alignment.normalize() * ALIGNMENT_WEIGHT
```

- Steer toward average direction of neighbors

2. **Cohesion** (Lines 245, 255-260):

```python
cohesion += other.position  # Sum all neighbor positions
center_mass = cohesion / total
cohesion_vec = center_mass - self.position
```

- Steer toward center of mass of neighbors

3. **Separation** (Lines 246-248, 262-263):

```python
diff = self.position - other.position
separation += diff.normalize() / d  # Inverse distance weighting
```

- Steer away from neighbors (stronger when closer)

Combined Effect (Line 265):

```
self.velocity += alignment + cohesion + separation

self.speed = self.base_speed + (total * FLOCK_SPEED_BOOST)
```

- All three forces are combined
- Speed increases with flock size

Drawing (Lines 270-275):

- Draws trail first

- Color fades with energy (low energy = darker green)
- Renders as a circle

4.6 Predator Class (Lines 277-349)

Purpose: Simpler agent focused on hunting

Initialization (Lines 278-280):

```
def __init__(self, x=None, y=None):

    super().__init__(x, y, PREDATOR_SPEED, PREDATOR_COLOR)

    self.vision_radius = PREDATOR_VISION_RADIUS  # 100 pixels (better
vision!)
```

Update Method (Lines 282-334):

1. **Early Checks** (Lines 283-291):

- Check if alive

- Update energy and cooldown

- If mating, stop moving

2. **Priority 1: Hunt Prey** (Lines 293-304):

```python

# Find nearest prey within vision radius

if nearest_prey:

    hunt_dir = (nearest_prey.position - self.position).normalize()

    self.velocity += hunt_dir * 0.4

```

- Always hunts if prey is visible

- Steers directly toward nearest prey

3. **Priority 2: Seek Mate** (Lines 306-324):

- Only if no prey visible AND energy > 80%

- Same mating logic as prey

4. **Priority 3: Obstacle Avoidance** (Line 327)
5. **Final Steps** (Lines 329-334):

- Normalize velocity

- Update position

- Check bounds

- Update trail

Drawing (Lines 336-349):

- Draws trail
- Renders as a **triangle** pointing in direction of movement
- Color fades with energy
- Uses rotation to orient triangle correctly

4.7 Simulation Class (Lines 356-595)

Purpose: Main controller managing the entire simulation

Initialization (Lines 357-372):

```
def __init__(self):

    # Create initial populations

    self.prey_list = [Prey() for _ in range(INITIAL_PREY)]

    self.predator_list = [Predator() for _ in range(INITIAL_PREDATORS)]

    self.food_list = [Food() for _ in range(INITIAL_FOOD)]

    self.obstacles = [Obstacle() for _ in range(INITIAL_OBSTACLES)]



    # State tracking

    self.running = True

    self.show_overlay = False
```

```
    # History arrays (unlimited length)

    self.history_prey = []

    self.history_predator = []

    self.history_prey_births = []

    self.history_predator_births = []

    self.timestep = 0
```

Spawn Child Method (Lines 374-388):

```
def spawn_child(self, parent1, parent2, agent_list, type_class):

    child = type_class()  # Create new agent

    child.position = (parent1.position + parent2.position) / 2  # Spawn
between parents

    child.energy = MAX_ENERGY  # Full energy

    agent_list.append(child)



    # Reset parent cooldowns

    parent1.reproduction_cooldown = REPRODUCTION_COOLDOWN_FRAMES

    parent2.reproduction_cooldown = REPRODUCTION_COOLDOWN_FRAMES

    parent1.mating_partner = None

    parent2.mating_partner = None
```

---

5. Main Simulation Loop

The `run()` method (Lines 390-490) contains the main game loop that runs at 60 FPS.

Loop Structure

1. Frame Timing (Line 394):

```
clock.tick(FPS)  # Wait to maintain 60 FPS
```

2. Clear Screen (Line 395):

```
screen.fill(BACKGROUND_COLOR)
```

3. Event Handling (Lines 397-423):

Processes keyboard input:

- **P**: Spawn new Prey
- **O**: Spawn new Predator
- **F**: Spawn new Food
- **B**: Spawn new Obstacle
- **1**: Toggle flocking on/off
- **2/3**: Increase/decrease food spawn rate
- **4/5**: Increase/decrease energy loss (difficulty)
- **G**: Show matplotlib graphs (pauses simulation)
- **H**: Toggle real-time graph overlay

4. Update Simulation (Lines 425-469):

a. **Increment Timestep** (Line 426):

```
self.timestep += 1
```

b. **Spawn Food** (Lines 430-432):

```
if random.random() < FOOD_SPAWN_CHANCE:
    self.food_list.append(Food())
```

c. **Update Prey** (Lines 434-451):

```
for prey in self.prey_list[:]:  # [:] creates copy to safely modify list
    prey.update(...)  # Run behavior logic

    if not prey.alive:
        self.prey_list.remove(prey)  # Remove dead prey
        continue

    # Check food collision
    for food in self.food_list[:]:
        if prey.position.distance_to(food.position) < prey.radius +
food.radius:
            prey.energy = min(MAX_ENERGY, prey.energy + FOOD_ENERGY_GAIN)
            self.food_list.remove(food)

    # Check reproduction (mating timer just finished)
    if prey.mating_timer == 1 and prey.mating_partner:
        if prey.mating_partner in self.prey_list:
            self.spawn_child(prey, prey.mating_partner, self.prey_list,
Prey)
```

d. **Update Predators** (Lines 453-469):

- Similar to prey update
- Checks for prey collision (catch distance = 10 pixels)
- Handles reproduction

5. Record History (Lines 471-475):

```
self.history_prey.append(len(self.prey_list))
```

```
self.history_predator.append(len(self.predator_list))

self.history_prey_births.append(self.prey_births)

self.history_predator_births.append(self.predator_births)
```

6. Rendering (Lines 477-487):

```
# Draw in order (background to foreground)

for obs in self.obstacles: obs.draw()

for food in self.food_list: food.draw()

for prey in self.prey_list: prey.draw()

for pred in self.predator_list: pred.draw()


self.draw_ui()  # Draw statistics overlay

if self.show_overlay:

    self.draw_realtime_graphs()  # Draw live graph


pygame.display.flip()  # Update screen
```

7. Cleanup (Lines 489-490):

```
self.save_graphs()  # Generate final graphs

pygame.quit()        # Close pygame
```

---

6. Key Algorithms and Behaviors

6.1 Collision Detection

Food-Prey Collision (Line 443):

```
if prey.position.distance_to(food.position) < prey.radius + food.radius:
```

- Uses Euclidean distance
- Collision when distance < sum of radii

Predator-Prey Collision (Line 462):

```
if predator.position.distance_to(prey.position) < 10:
```

- Fixed catch distance of 10 pixels
- Simpler than radius-based (predators are triangles)

6.2 Velocity Normalization

After all steering forces are applied, velocity is normalized:

```
if self.velocity.length() > 0:
    self.velocity = self.velocity.normalize()
```

- Ensures consistent speed regardless of number of forces
- Only direction matters, speed is controlled separately

6.3 Boundary Bouncing

```
if self.position.x < 0 or self.position.x > WIDTH:
    self.velocity.x *= -1  # Reverse X direction
    self.position.x = max(0, min(self.position.x, WIDTH))  # Clamp position
```

- Agents bounce off walls rather than wrapping around

6.4 Mating State Machine

1. **Seeking**: Agent looks for eligible partner
2. **Approaching**: Moves toward partner
3. **Mating**: Both agents stop moving for `MATING_DURATION_FRAMES`
4. **Reproduction**: Child spawns when timer reaches 1
5. **Cooldown**: Both parents enter cooldown period

---

7. User Controls and Features

7.1 Real-time Parameter Adjustment

Flocking Toggle (Key 1):

```
FLOCKING_ENABLED = not FLOCKING_ENABLED
```

- Instantly enables/disables Boids behavior
- Useful for comparing flocking vs. individual behavior

Food Spawn Rate (Keys 2/3):

```
FOOD_SPAWN_CHANCE = min(1.0, FOOD_SPAWN_CHANCE + 0.01)  # Increase

FOOD_SPAWN_CHANCE = max(0.0, FOOD_SPAWN_CHANCE - 0.01)  # Decrease
```

- Adjusts resource abundance
- Higher = easier survival for prey

Energy Loss Rate (Keys 4/5):

```
ENERGY_LOSS_RATE *= 1.1  # Harder (10% increase)

ENERGY_LOSS_RATE *= 0.9  # Easier (10% decrease)
```

- Multiplicative adjustment
- Higher = agents die faster

7.2 Manual Spawning

All spawn keys create entities at random positions:

- **P**: Adds Prey (helps struggling population)
- **O**: Adds Predator (increases predation pressure)
- **F**: Adds Food (emergency food supply)
- **B**: Adds Obstacle (creates environmental complexity)

---

8. Data Visualization

8.1 Real-time Graph Overlay (Lines 505-549)

Purpose: Show live population trends without pausing simulation

Implementation:

```
def draw_realtime_graphs(self):
    # Create semi-transparent background
    s = pygame.Surface((graph_w, graph_h))
    s.set_alpha(128)  # 50% transparency

    # Normalize data to fit graph height
    max_pop = max(max(self.history_prey), max(self.history_predator), 1)

    # Show last 500 frames
    start_idx = max(0, len(self.history_prey) - 500)

    # Draw lines using pygame.draw.lines
    pygame.draw.lines(screen, PREY_COLOR, False, points_prey, 2)
    pygame.draw.lines(screen, PREDATOR_COLOR, False, points_pred, 2)
```

Features:

- Shows last 500 frames (sliding window)
- Auto-scales to fit data range
- Drawn with Pygame for performance (matplotlib too slow)

8.2 Matplotlib Graphs

Show Graphs (Lines 579-595):

- Called when 'G' is pressed
- Shows full history in popup window
- Two subplots: Population and Birth Events

Save Graphs (Lines 551-577):

- Called when simulation exits
- Saves high-quality PNG to `Graphs_For_Project_one/Simulation_graphs.png`
- Creates directory if needed

Graph Content:

1. **Population History**: Prey (green) and Predator (red) counts over time
2. **Birth Events**: Number of births per timestep for each species

---

9. Running the Simulation

9.1 Starting the Simulation

```
python Project_One.py
```

The simulation window will open with:

- 50 green prey circles
- 3 red predator triangles
- 20 yellow food dots
- 5 gray obstacles
- Statistics overlay in top-left

9.2 Complete Control Reference

| Key | Function | Effect |

| --- | --- | --- |

| **P** | Spawn Prey | Adds 1 prey at random location |

| **O** | Spawn Predator | Adds 1 predator at random location |

| **F** | Spawn Food | Adds 1 food pellet |

| **B** | Spawn Obstacle | Adds 1 obstacle |

| **1** | Toggle Flocking | Enable/disable Boids behavior |

| **2** | Increase Food Rate | +0.01 to spawn chance (max 1.0) |

| **3** | Decrease Food Rate | -0.01 to spawn chance (min 0.0) |

| **4** | Increase Difficulty | Energy loss × 1.1 (harder) |

| **5** | Decrease Difficulty | Energy loss × 0.9 (easier) |

| **H** | Toggle Overlay | Show/hide real-time graph |

| **G** | Show Graphs | Pause and display matplotlib graphs |

| **X** (Close) | Exit | Save graphs and quit |

9.3 Visual Indicators

- **Green Circles**: Prey (size = 4 pixels)

- Bright green = high energy

- Dark green = low energy (near death)

- Trail shows recent movement path

- **Red Triangles**: Predators (size = 10×5 pixels)

- Point in direction of movement

- Bright red = high energy

- Dark red = low energy

- Trail shows hunting path

- **Yellow Dots**: Food (size = 3 pixels)

- Static until consumed

- Provides 30 energy when eaten

- **Gray Circles**: Obstacles (size = 30 pixels)

- Static barriers

- Agents avoid within 60 pixels

9.4 Understanding Population Dynamics

Typical Behavior:

1. **Initial Growth**: Prey reproduce quickly (high food, low predation)
2. **Predator Response**: Predators catch up, prey population drops
3. **Oscillation**: Classic Lotka-Volterra cycles
4. **Stability**: System may reach equilibrium or continue oscillating

Extinction Scenarios:

- **Prey Extinction**: If predators too efficient or food too scarce
- **Predator Extinction**: If prey too fast or too few predators initially

Population Explosion:

- Rare due to energy constraints
- Food scarcity limits prey growth
- Predator starvation limits predator growth

9.5 Experimentation Tips

1. **Test Flocking**: Toggle with '1' to see difference in prey behavior
2. **Adjust Food**: Use 2/3 to see how resource abundance affects stability
3. **Change Difficulty**: Use 4/5 to make survival easier/harder
4. **Watch Graphs**: Press 'H' to see real-time trends
5. **Manual Intervention**: Use P/O/F to rescue struggling populations

---

10. Code Flow Summary

Execution Order (Per Frame)

1. **Event Processing**: Handle keyboard input
2. **Food Spawning**: Random chance to add food
3. **Prey Updates**:

- Behavior logic (flee/eat/mate/flock)

- Food collision detection

- Reproduction check

4. **Predator Updates**:

- Behavior logic (hunt/mate)

- Prey collision detection

- Reproduction check

5. **History Recording**: Store population counts
6. **Rendering**: Draw all entities, UI, and graphs
7. **Display**: Update screen

Key Design Patterns

- **Inheritance**: Prey and Predator inherit from Agent
- **Composition**: Simulation contains lists of entities
- **State Machine**: Mating uses timer-based state transitions
- **Priority System**: Prey behaviors are prioritized (flee > eat > mate > flock)
- **Separation of Concerns**: Each class handles its own logic

---

11. Mathematical Concepts

11.1 Vector Mathematics

The simulation uses `pygame.math.Vector2` for 2D vectors:

- **Position**: `(x, y)` coordinates
- **Velocity**: Direction and magnitude
- **Normalization**: Convert to unit vector (length = 1)
- **Distance**: Euclidean distance between points

11.2 Boids Algorithm Mathematics

Alignment Force:

```
alignment = average(neighbor_velocities)
```

Cohesion Force:

```
center_of_mass = average(neighbor_positions)
```

```
cohesion = center_of_mass - self.position
```

Separation Force:

```
separation = sum((self.position - neighbor_position) / distance²)
```

11.3 Energy System Mathematics

```
energy(t) = energy(t-1) - ENERGY_LOSS_RATE

if energy <= 0: agent dies

if collision with food: energy += FOOD_ENERGY_GAIN
```

11.4 Population Dynamics

The simulation approximates the **Lotka-Volterra equations**:

- Prey growth rate depends on food availability
- Predator growth rate depends on prey availability
- Both have death rates (energy loss)
- Results in oscillating populations

Conclusion

This simulation demonstrates how simple rules can create complex, emergent behavior. By combining:

- Energy-based survival
- Reproduction mechanics
- Flocking behavior
- Predator-prey interactions

We create a dynamic ecosystem that exhibits realistic population dynamics. The code is structured for easy modification and experimentation, making it an excellent learning tool for understanding agent-based modeling and biological simulations.

File: `Project_One.py`

Total Lines: 599

Classes: 5 (Food, Obstacle, Agent, Prey, Predator, Simulation)

Key Algorithms: Boids Flocking, Priority-based Behavior, Collision Detection